

Techniki Mikroprocesorowe

Sprawozdanie – Laboratorium 2

Temat: Program wbudowany dla mikrokontrolera avr.

Obsługa wyświetlacza 7-segmentowego

Autor: Kamil Sitarski

Numer albumu: [REDACTED]

Grupa dziekańska: [REDACTED]

Data wykonania ćwiczenia: 12.05.2026r.

Zadanie 1

1. Opis zadania

Celem zadania była definicja symboli cyfr od 0 do 9 i wyświetlenie swojego roku urodzenia. Czas opóźnienia trzeba było tak dobrać, aby uzyskać stabilny stan wyświetlanej treści na wyświetlaczu.

2. Opis kodu

Program realizuje wyświetlanie poprzez szybkie przełączanie zasilania pomiędzy kolejnymi cyframi wyświetlacza, co przy odpowiedniej częstotliwości tworzy złudzenie ciągłego świecenia wszystkich znaków.

Definicja znaków:

Wewnątrz funkcji main zdefiniowano lokalną tablicę, która przechowuje wzorce bitowe dla cyfr od 0 do 9. Przyjęto że PA0 odpowiada segmentowi a.

```
unsigned char cyfry[10] = {  
    0b00111111, //cyfra 0  
    0b00000110, //cyfra 1  
    0b01011011, //cyfra 2  
    0b01001111, //cyfra 3  
    0b01100110, //cyfra 4  
    0b01101101, //cyfra 5  
    0b01111101, //cyfra 6  
    0b00000111, //cyfra 7  
    0b01111111, //cyfra 8  
    0b01101111 //cyfra 9  
  
};
```

- **Konfiguracja portów:**

DDRA = 0xFF; //Konfiguracja portu A: Wszystkie piny na wyjście (przesyłają dane do segmentów)

DDRB = 0x0F; //Konfiguracja portu B: Ustawiono cztery najmłodsze piny na wyjścia do sterowania tranzystorami do wyboru kolumny

- **Algorytm multipleksowania (Pętla while(1)):**

Program w każdym cyklu wykonuje cztery powtarzalne kroki dla każdej cyfry roku (rok: 2005).

Przykład (cyfra 2):

```
PORTB = ~(1<<0);    //wybór kolumny
PORTA = ~cyfry[2];  //wysłanie danych
_delay_ms(2);        //wstrzymanie programu na 2 milisekundy
```

Cykl dla każdej cyfry roku składa się z:

1. **Wyboru kolumny:**

Za pomocą instrukcji *PORTB* = ~(1 << x); aktywowany jest konkretny wyświetlacz.
Użyto negacji bitowej, ponieważ sterowanie jest stanem niskim (wspólna anoda)

2. **Wysłania danych:**

Instrukcja *PORTA* = ~cyfry[n]; wysyła zanegowany wzorzec cyfry na segmenty.

3. **Opóźnienia:**

Funkcja *_delay_ms(2)*; wstrzymuje program na 2 milisekundy, pozwalając diodom LED na zaświecenie się z odpowiednią jasnością.

3. Kod źródłowy programu

```
/*
 * Laboratorium2.c
 *
 * Created: 12.05.2026 16:09:00
 * Author : student_pl
 */
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    unsigned char cyfry[10] = {
        0b00111111,
        0b00000110,
        0b01011011,
        0b01001111,
        0b01100110,
        0b01101101,
        0b01111101,
        0b00000111,
        0b01111111,
        0b01101111
    };
    DDRA = 0xFF;
    DDRB = 0x0F;

    while (1)
    {
        PORTB = ~(1<<0);
        PORTA = ~cyfry[2];
        _delay_ms(2);

        PORTB = ~(1<<1);
        PORTA = ~cyfry[0];
        _delay_ms(2);

        PORTB = ~(1<<2);
        PORTA = ~cyfry[0];
        _delay_ms(2);

        PORTB = ~(1<<3);
        PORTA = ~cyfry[5];
        _delay_ms(2);
    }
}
```

Zadanie 2

1. Opis zadania

Zadanie polegało na zaprojektowaniu licznika, który w pętli nieskończonej wyświetla wartości od 2 do 142, zwiększając je o 7 w każdym kroku. Kluczowymi wymaganiami były: zmiana wyświetlanej liczby co ok. 0,5 sekundy oraz implementacja mechanizmu wygaszania zer nieznaczących (brak wyświetlania zer przed właściwą liczbą, np. "2" zamiast "002").

2. Opis kodu

- **Rozbijanie liczby na cyfry:**

Wewnątrz funkcji `wyswietl_liczbe(int liczba)` zastosowano operacje arytmetyczne do wyodrębnienia poszczególnych cyfr z liczby całkowitej:

- `uint8_t jednosci = liczba%10;` // reszta z dzielenia przez 10 daje cyfrę jedności.
- `uint8_t dziesiatki = (liczba / 10) % 10;` // dzielenie całkowite przez 10 przesuwa liczbę, a modulo 10 wyodrębnia cyfrę dziesiątek.
- `uint8_t setki = (liczba / 100) % 10;` // analogicznie wyodrębniana jest cyfra setek.

- **Wygaszanie zer nieznaczących:**

Zrealizowano to za pomocą instrukcji warunkowych `if(liczba > 9)` oraz `if(liczba > 99)`. Program aktywuje odpowiednią kolumnę (PORTB) i wysyła dane na segmenty (PORTA) tylko wtedy, gdy liczba faktycznie posiada cyfrę na danej pozycji dziesiątnej. Dzięki temu dla liczby mniejszej niż 100 wyświetlacz setek pozostaje zgaszony.

- **Timing i stabilizacja obrazu:**

Główna pętla `for(int i=0; i<83; i++)` wewnątrz funkcji wykonuje 83 iteracje, z których każda trwa ~6 ms (3 pozycje po 2 ms opóźnienia), daje łączny czas wyświetlania jednej liczby równy ~0,5 s ($83 \cdot 6ms = 498ms$). Zapewnia to zgodną z poleceniem prędkość odliczania.

- **Logika sprzętowa:**

Użycie negacji bitowej (~) przy sterowaniu portami A i B wskazuje na współpracę mikrokontrolera z wyświetlaczem ze wspólną anodą oraz tranzystorami kluczującymi typu PNP.

3. Kod źródłowy programu

```
/*
 * Laboratorium2.c
 *
 * Created: 12.05.2026 16:09:00
 * Author : student_pl
 */
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>

unsigned char cyfry[10] = {
    0b00111111,
    0b00000110,
    0b01011011,
    0b01001111,
    0b01100110,
    0b01101101,
    0b0111101,
    0b0111101,
    0b00000111,
    0b01111111,
    0b01101111
};

void wyswietl_liczbe(int liczba){
    uint8_t jednosci = liczba%10;
    uint8_t dziesiatki = (liczba/10)%10;
    uint8_t setki = (liczba/100)%10;

    for(int i=0;i<83;i++){
        PORTB = ~(1<<3);
        PORTA = ~cyfry[jednosci];
        _delay_ms(2);

        if(liczba>9){
            PORTB = ~(1<<2);
            PORTA = ~cyfry[dziesiatki];

        }
        _delay_ms(2);

        if(liczba>99){
            PORTB = ~(1<<1);
            PORTA = ~cyfry[setki];

        }
        _delay_ms(2);

    }
}

int main(void)
{
    DDRA = 0xFF;
    DDRB = 0x0F;
    while (1)
    {
        for(int i=2;i<=142;i+=7){
            wyswietl_liczbe(i);

        }

    }
}
```

Zadanie 3

1. Opis zadania

Zadanie polegało na stworzeniu dynamicznej animacji na wyświetlaczu 7-segmentowym, polegającej na sekwencyjnym zapalaniu poszczególnych segmentów na różnych pozycjach wyświetlacza. Celem było stworzenie płynnego efektu ruchu ("węża") krążącego po obwodzie wszystkich czterech cyfr.

2. Opis kodu

- **Definicja klatek animacji:**

W tablicy [znaki\[10\]](#) zdefiniowano niestandardowe wzorce bitowe. W przeciwieństwie do poprzednich zadań, bity te nie tworzą cyfr, lecz zapalają pojedyncze segmenty lub ich pary (np. 0b00000001 to segment a, 0b00000011 to segmenty a i b jednocześnie).

- **Logika sterowania kolumnami (DDRB):**

Cechą wyróżniającą ten kod jest wykorzystanie rejestru kierunku DDRB do wyboru aktywnej kolumny. Zamiast zmieniać stany logiczne na portach, program dynamicznie przełącza piny Portu B między trybem wejścia (wysoka impedancja – kolumna zgaszona) a wyjściem ($DDRB = (1 \ll n)$). Jest to zaawansowana metoda zapobiegająca "duchom" (powidokom) na sąsiednich wyświetlaczach.

- **Struktura animacji (Zmienna i oraz j):**

- **Zmienna j (Pętla wewnętrzna):**

Odpowiada za czas trwania jednej "klatki" animacji. Wykonanie 200 powtórzeń z krótkimi opóźnieniami sprawia, że dany układ segmentów jest widoczny dla oka przez ułamek sekundy.

- **Zmienna i (Licznik klatek):**

Steruje postępem animacji. Instrukcje warunkowe `if(i == ...)` definiują, co dzieje się w każdym z 12 etapów cyklu.

- **Realizacja ruchu (Analiza sekwencji animacji)**

Sercem algorytmu odpowiedzialnego za efekt wizualny „węża” jest kombinacja licznika klatek (zmienna i), pętli opóźniającej (zmienna j) oraz rozbudowanej struktury instrukcji warunkowych if. Dynamiczny ruch uzyskano poprzez jednoczesną, skoordynowaną manipulację danymi wysyłanymi na segmenty oraz kluczowaniem zasilania poszczególnych cyfr.

Szczegółowa analiza fizycznego zachowania programu w poszczególnych fazach cyklu:

1) Faza początkowa – Rozbieg (stany licznika $i = 0$ oraz $i = 1$):

- Gdy program napotyka instrukcję warunkową `if(i==0)`, realizowane jest klasyczne multipleksowanie programowe, ale dla jednego, konkretnego wzorca bitowego `znaki[0]` (zapalony pojedynczy segment a). Poprzez błyskawiczne sekwencyjne przełączanie wyjść DDRB (kolejno piny 0, 1 i 2) z opóźnieniem 2 ms, na trzech pierwszych wyświetlaczach od prawej strony zapala się górna kreska. Ludzkie oko widzi ten efekt jako jednoczesne świecenie trzech segmentów.
- Przy kolejnym stanie licznika, kiedy spełniony jest warunek `if(i==1)`, następuje przesunięcie tego potrójnego paska o jedną pozycję w lewo (aktywowane są piny 1, 2 i 3 rejestru DDRB). Daje to pierwszy efekt ruchu – linia „wypełza” z prawej strony i przesuwa się w lewo.

2) Faza wejścia w zakręt i ruchu pionowego (stany licznika $i = 2$ do $i = 5$):

- Gdy program wchodzi w instrukcję `if(i==2)`, ruch w lewo zostaje zatrzymany na skrajnej, czwartej cyfrze (`DDRB = (1 << 3)`). Następuje tu zmiana znaku z `znaki[0]` na `znaki[1]`. Wzorzec ten dokłada do świecącego segmentu a również pionowy segment f (lub b w zależności od fizycznego podłączenia płytki). Punkt zaczyna „skręcać” i schodzić w dół po krawędzi wyświetlacza.
- W stanach licznika spełniających warunek `if(i>=3 && i<=4)`, zasilanie jest na stałe zablokowane na czwartej pozycji (`DDRB = (1 << 3)`). Animacja realizowana jest wyłącznie poprzez zmianę indeksu tablicy `znaki[i-1]`. Powoduje to sekwencyjne gaszenie górnych segmentów i zapalanie dolnych, co wywołuje efekt fizycznego przemieszczania się pojedynczej kreski w dół wzdłuż lewej krawędzi całego modułu wyświetlaczy.
- Przy stanie licznika opisanym instrukcją `if(i==5)`, punkt dociera do dolnego zakrętu czwartej cyfry (`znaki[4]` i `znaki[5]`), przygotowując się do zmiany kierunku ruchu.

3) Faza powrotu dołem (stany licznika $i = 6$ oraz $i = 7$):

- W warunku `if(i==6)` wąż rozpoczyna ruch powrotny w lewą stronę, wykorzystując segmenty dolne (segment d). Podobnie jak w pierwszej fazie, program szybko przełącza rejestr DDRB pomiędzy pinami 3, 2 i 1, utrzymując na porcie A stały wzorzec `znaki[5]`. Wywołuje to efekt ciągłej dolnej linii pod trzema cyframi.
- Kiedy mikrokontroler przechodzi do instrukcji `if(i==7)`, dolna linia przesuwa się w prawo, aktywując pozycje tranzystorów sterowanych przez piny 2, 1 oraz 0.

4) Faza pionowego powrotu i zamknięcia pętli (stany licznika $i = 8$ do $i = 11$):

- Przy stanie licznika w instrukcji `if(i==8)`, punkt dociera do skrajnej prawej cyfry (`DDRB = (1 << 0)`) i zaczyna piąć się w górę po jej prawej krawędzi, płynnie zmieniając indeks na `znaki[6]`.
- Instrukcje warunkowe `if(i==9)` oraz `if(i==10)` realizują ruch pionowo w górę na pierwszej cyfrze za pomocą wzorców `znaki[7]` oraz `znaki[8]`. Port B pozostaje niezmienny, pracując cały czas na najmłodszym bicie.
- W ostatnim kroku sekwencji, przy instrukcji `if(i==11)`, punkt na pierwszej cyfrze przyjmuje wzorec `znaki[9]` (lewy górny róg), po czym następuje szybkie przełączenie na pozycję drugą (`DDRB = (1 << 1)`), gdzie wyświetlany jest stan początkowy `znaki[0]` (segment a).

Po wykonaniu wszystkich instrukcji warunkowych, sprawdzenie `if(i==12)` resetuje zmienną sterującą do zera ($i=0$). Cała sekwencja zaczyna się od nowa w pętli nieskończonej `while(1)`, tworząc dla obserwatora złudzenie płynnego, zapętłonego ruchu punktu po obwodzie całego modułu wyświetlaczy LED. Modulacja czasów delay (od 2 ms do 6 ms) w poszczególnych blokach `if` została dobrana eksperymentalnie, aby zrównoważyć jasność świecenia diod w momentach, gdy świeci kilka segmentów jednocześnie, w stosunku do momentów, gdy świeci tylko jedna linia.

- **Przebieg czasowy i płynność:**

Kluczowym elementem algorytmu jest precyzyjnie dobrany czas trwania każdego stanu licznika. Aby animacja była stabilna, a jasność wyświetlacza jednolita w każdym punkcie, łączny czas trwania pojedynczej iteracji w każdym bloku warunkowym został zbilansowany do dokładnie 6 ms. Zależnie od tego, ile pozycji wyświetlacza jest aktywowanych w danym kroku, czas ten jest sumowany z mniejszych podokresów:

- W blokach, gdzie aktywowane są kolejno trzy pozycje (ruch po obwodzie), program wykonuje trzy podokresy po 2 ms ($3 \cdot 2 \text{ ms} = 6 \text{ ms}$).
- W blokach, gdzie aktywowane są dwie pozycje (zakręty), program wykonuje dwa podokresy po 3 ms ($2 \cdot 3 \text{ ms} = 6 \text{ ms}$).
- W blokach, gdzie punkt porusza się tylko na jednej pozycji (ruch pionowy), stosowane jest jedno ciągłe opóźnienie wynoszące 6 ms.

Dzięki temu, że każdy stan licznika trwa w jednej iteracji pętli dokładnie 6 ms, wykonanie całej wewnętrznej pętli `for` (200 iteracji) zajmuje zawsze około 1,2 sekundy ($200 \cdot 6 \text{ ms} = 1200 \text{ ms}$), co zapewnia idealnie równomierne tempo animacji.

3. Kod źródłowy programu

```
/*
 * Laboratorium2.c
 *
 * Created: 12.05.2026 16:09:00
 * Author : student_pl
 */
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>

unsigned char znaki[10] = {
    0b00000001,
    0b00000011,
    0b00000111,
    0b00001110,
    0b00001100,
    0b00001000,
    0b00011000,
    0b00111000,
    0b00110001,
    0b00100001,
};

int main(void)
{
    DDRA = 0xFF;
    DDRB = 0x0F;

    int i = 0;
    while (1)
    {
        for(int j=0;j<=200;j++){
            if(i==0){
                DDRB = (1<<0);
                DDRA = znaki[0];
                _delay_ms(2);

                DDRB = (1<<1);
                DDRA = znaki[0];
                _delay_ms(2);

                DDRB = (1 <<2);
                DDRA = znaki[0];
                _delay_ms(2);
            }

            if(i==1){
                DDRB = (1 << 1);
                DDRA = znaki[0];
                _delay_ms(2);

                DDRB = (1 << 2);
                DDRA = znaki[0];
                _delay_ms(2);
            }
        }
        i++;
    }
}
```

```

        DDRB = (1 << 3);
        DDRA = znaki[0];
        _delay_ms(2);
    }

    if(i==2){
        DDRB = (1<< 2);
        DDRA = znaki[0];
        _delay_ms(3);

        DDRB = (1 <<3);
        DDRA = znaki[1];
        _delay_ms(3);
    }

    if(i>=3 && i<=4){
        DDRB = (1<<3);
        DDRA = znaki[i-1];
        _delay_ms(6);
    }

    if(i==5){
        DDRB = (1<< 3);
        DDRA = znaki[4];
        _delay_ms(3);

        DDRB = (1 << 2);
        DDRA = znaki[5];
        _delay_ms(3);
    }

    if(i==6){
        DDRB = (1<< 3);
        DDRA = znaki[5];
        _delay_ms(2);

        DDRB = (1<< 2);
        DDRA = znaki[5];
        _delay_ms(2);

        DDRB = (1 << 1);
        DDRA = znaki[5];
        _delay_ms(2);
    }

    if(i==7){
        DDRB = (1 << 2);
        DDRA = znaki[5];
        _delay_ms(2);

        DDRB = (1 << 1);
        DDRA = znaki[5];
        _delay_ms(2);

        DDRB = (1 << 0);
        DDRA = znaki[5];
        _delay_ms(2);
    }

```

```

        if(i==8){
            DDRB = (1<< 1);
            DDRA = znaki[5];
            _delay_ms(3);

            DDRB = (1 << 0);
            DDRA = znaki[6];
            _delay_ms(3);
        }

        if(i==9){
            DDRB = (1<< 0);
            DDRA = znaki[7];
            _delay_ms(6);
        }

        if(i==10){
            DDRB = (1<< 0);
            DDRA = znaki[8];
            _delay_ms(6);
        }

        if(i==11){
            DDRB = (1<< 0);
            DDRA = znaki[9];
            _delay_ms(3);

            DDRB = (1 << 1);
            DDRA = znaki[0];
            _delay_ms(3);
        }
    }
    i++;
    if(i==12){i=0;}
}
}

```

Wnioski

1. **Efektywność multipleksowania programowego:** Przeprowadzone ćwiczenia potwierdziły, że technika multipleksowania jest kluczowym rozwiązaniem inżynierskim pozwalającym na znaczną oszczędność linii sterujących mikrokontrolera. Dzięki naprzemiennemu aktywowaniu kolumn z częstotliwością niezauważalną dla ludzkiego oka (wykorzystując zjawisko bezwładności wzroku), sterowanie czterema cyframi (łącznie 32 segmentami) powiodło się przy użyciu zaledwie 12 pinów procesora (8 linii danych Portu A oraz 4 linie adresowe Portu B).
2. **Ścisły reżim czasowy a jakość obrazu:** Stabilność prezentowanych danych oraz brak efektu migotania zależą bezpośrednio od dobranych opóźnień programowych (`_delay_ms`). Ćwiczenia pokazały że czas 2 ms mieści się w przedziale optymalnego czasu wyświetlania. Zbyt długie opóźnienie powoduje widoczne, uciążliwe migotanie diod, natomiast zbyt krótkie uniemożliwia pełne otwarcie złącza tranzystora, drastycznie ograniczając jasność świecenia.
3. **Rola zbilansowanego budżetu czasowego w animacji:** Na przykładzie Zadania 3 wykazano, że zachowanie stałego czasu trwania każdego stanu licznika (zbilansowanie każdej gałęzi instrukcji warunkowej `if` do sumarycznych 6 ms) jest kluczowe dla zachowania jednolitej jasności świecenia diod LED. Bez rygorystycznego podziału czasu, segmenty zapalane pojedynczo świeciłyby znacznie jaśniej niż te świecące w grupach, zaburzając płynność i estetykę animacji.
4. **Wykorzystanie rejestrów kierunku (DDR) do eliminacji powidoków:** Sterowanie zasilaniem kolumn wyświetlacza poprzez przełączanie rejestru kierunku (DDRB), zamiast modyfikacji rejestru wyjściowego (PORTB), okazało się zaawansowaną i wysoce skuteczną metodą gaszenia nieaktywnych sekcji. Wprowadzanie pinów w stan wysokiej impedancji (tryb wejścia) całkowicie odcinało przepływ prądu, skutecznie eliminując zjawisko tzw. „duchów” (słabego świecenia sąsiednich cyfr podczas przełączania danych).
5. **Uniwersalność matrycy 7-segmentowej:** Realizacja ostatniego zadania udowodniła, że wyświetlacz 7-segmentowy, poza swoim podstawowym przeznaczeniem alfanumerycznym, może służyć jako prosty interfejs graficzny do wyświetlania dynamicznych sygnałów statusowych lub prostych animacji diagnostycznych w systemach wbudowanych.